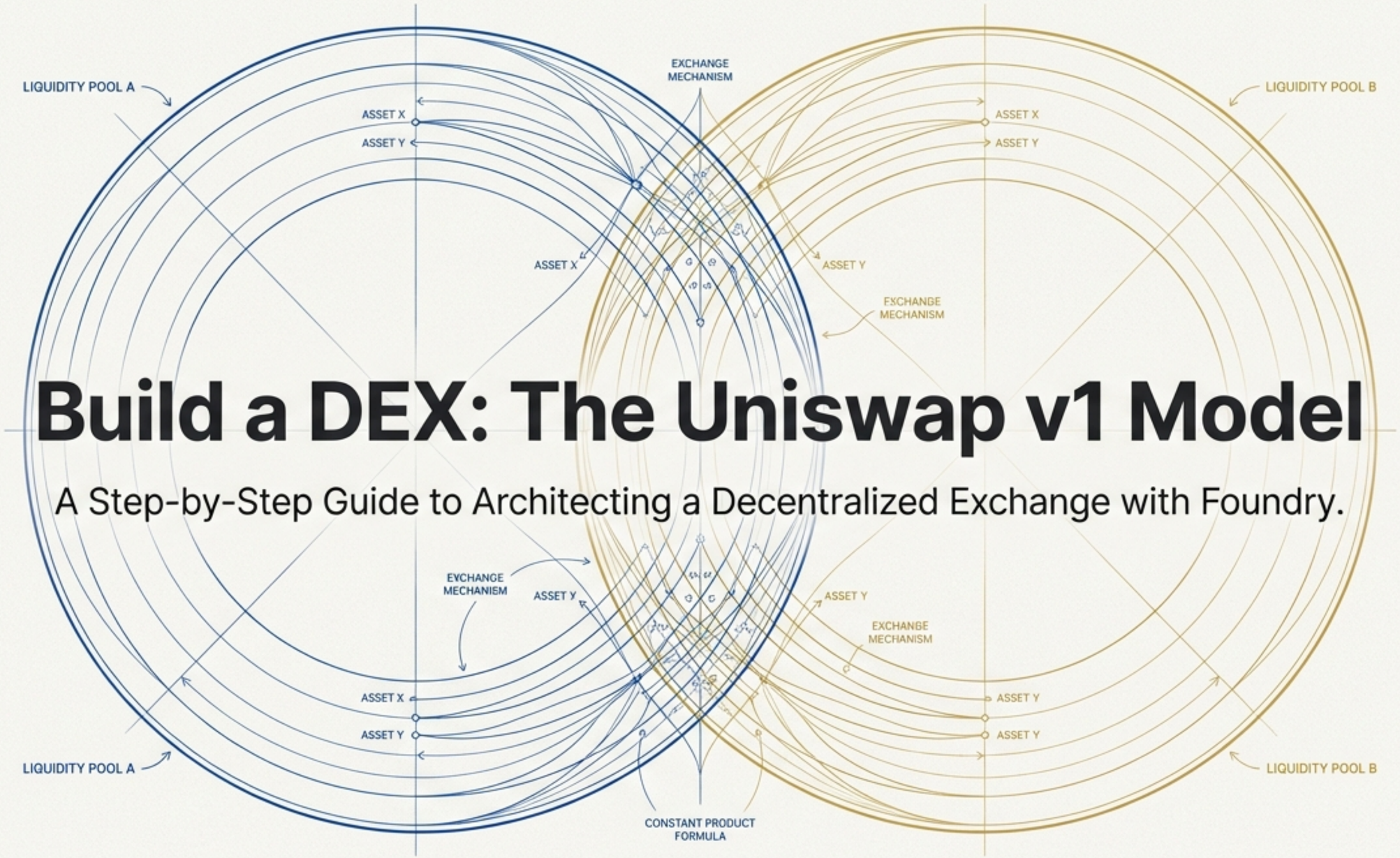


Build a DEX: The Uniswap v1 Model

A Step-by-Step Guide to Architecting a Decentralized Exchange with Foundry.

The Challenge: Enabling Trustless Swaps

Our Mission

Build an automated market maker (AMM) from scratch that allows users to seamlessly swap ETH for any ERC-20 token. This will be based on the foundational mathematics of Uniswap v1.

Core Requirements

- Enable ETH <> TOKEN swaps.
- Implement a 1% fee on all swaps.
- Issue Liquidity Provider (LP) tokens to represent a user's share of the pool.
- Allow LPs to add liquidity and later burn their LP tokens to reclaim their assets.

Our Toolkit

Foundry

For smart contract development and testing.

Solidity

The language for our smart contracts.

OpenZeppelin

For the standard ERC-20 contract implementation.

Step 1: Environment & Tooling Configuration

Initialize Project

```
# Create project directory
mkdir dex-app && cd dex-app

# Initialize Foundry project
forge init foundry-app
cd foundry-app

# Install OpenZeppelin contracts dependency
forge install OpenZeppelin/openzeppelin-
contracts

# Configure import remappings
forge remappings > remappings.txt
```

Configure Secrets

Create a `.env` file in the `foundry-app` folder. These variables are crucial for deploying and verifying your contracts on a testnet like Sepolia.

```
PRIVATE_KEY="..."
RPC_URL="..."
ETHERSCAN_API_KEY="..."
```

SECURITY ADVISORY

Always use a private key from an account that contains only testnet funds. Never commit your `.env` file to version control.

Step 2: Crafting the ERC-20 `Token.sol`

Before we can build an exchange, we need a tradeable asset. This contract serves as our standard ERC-20 token, which we will call 'Token' (TKN).

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.25;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract Token is ERC20 {
    // Initialize contract with 1 million tokens minted to the creator
    constructor() ERC20("Token", "TKN") {
        _mint(msg.sender, 1000000 * 10 ** decimals());
    }
}
```

Upon deployment, 1 million TKN are minted directly to your address, giving you the initial supply to provide liquidity.

Step 3: Designing the Exchange Contract

This contract is the heart of our DEX. It serves a crucial dual purpose:

1. It is the **liquidity pool**, holding the reserves of both ETH and our TKN token.
2. It is **also an ERC-20 contract itself**, responsible for minting and burning LP (Liquidity Provider) tokens.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.25;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract Exchange is ERC20 {
    address public tokenAddress;

    // Future function ali on from address tokens
    constructor(address token) ERC20("ETH TOKEN LP Token", "lpETHTOKEN") {
      require(token != address(0), "Token address passed is a null address");
      tokenAddress = token;
    }

    // Future functions will go here
}
```

The Exchange *is* an ERC-20. This allows it to mint its own unique LP tokens to liquidity providers.

Function Deep Dive: `addLiquidity`

The Problem

How do we enable users to deposit funds (ETH and TKN) and receive a proportional share of the pool, represented by LP tokens?

Initial Liquidity

Condition: The exchange has no token reserves (`tokenReserveBalance == 0`).

Action: The first depositor sets the initial exchange rate. We simply mint them an amount of LP tokens equal to the amount of ETH they deposited (`msg.value`).

Adding to an Existing Pool

Condition: The exchange already has reserves.

Action: The user must deposit ETH and TKN at the current ratio. The amount of LP tokens minted is proportional to their contribution relative to the total supply. (`lpTokensToMint = (totalSupply() * msg.value) / ethReservePrior`)

Code Implementation: `addLiquidity`

```
function addLiquidity(uint256 amountOfToken) public payable returns (uint256) {
    uint256 lpTokensToMint;
    uint256 ethReserveBalance = address(this).balance;
    uint256 tokenReserveBalance = getReserve(); // getReserve() returns ERC20(tokenAddress).balanceOf(address(this))
    ERC20 token = ERC20(tokenAddress);

    // SCENARIO 1: Initial Liquidity
    if (tokenReserveBalance == 0) {
        token.transferFrom(msg.sender, address(this), amountOfToken);
        lpTokensToMint = ethReserveBalance; // Based on msg.value
        _mint(msg.sender, lpTokensToMint);
        return lpTokensToMint;
    }

    // SCENARIO 2: Adding to Existing Pool
    uint256 ethReservePriorToFunctionCall = ethReserveBalance - msg.value;
    uint256 minTokenAmountRequired = (msg.value * tokenReserveBalance) / ethReservePriorToFunctionCall;
    require(amountOfToken >= minTokenAmountRequired, "Insufficient amount of tokens provided");
    token.transferFrom(msg.sender, address(this), minTokenAmountRequired);
    lpTokensToMint = (totalSupply() * msg.value) / ethReservePriorToFunctionCall;
    _mint(msg.sender, lpTokensToMint);

    return lpTokensToMint;
}
```

Handles the first liquidity deposit, setting the initial rate.

Calculates the required token deposit and proportional LP tokens for subsequent deposits.

Function Deep Dive: `removeLiquidity`

The Problem

How do liquidity providers redeem their LP tokens to reclaim their proportional share of the underlying ETH and TKN assets?

The Solution Logic

This function is the reverse of `addLiquidity`. A user specifies how many LP tokens they wish to "burn." In return, the contract calculates and transfers their proportional share of the current ETH and TKN reserves back to them.

The Core Formulas

$$\text{ethToReturn} = \frac{(\text{ethReserveBalance} * \text{amountOfLPTokens})}{\text{lpTokenTotalSupply}}$$

$$\text{tokenToReturn} = \frac{(\text{getReserve}() * \text{amountOfLPTokens})}{\text{lpTokenTotalSupply}}$$

Conceptual Code

```
// function removeLiquidity(uint256 amountOfLPTokens) ...
// ...
// 1. Calculate ethToReturn and tokenToReturn using the formulas above.
// 2. Burn the user's LP tokens:
_burn(msg.sender, amountOfLPTokens);
// 3. Transfer the assets back to the user:
payable(msg.sender).transfer(ethToReturn);
ERC20(tokenAddress).transfer(msg.sender, tokenToReturn);
// ...
```

The AMM Core: `getOutputAmountFromSwap`

The Problem

Given an input amount of one asset, how much of the other asset should the user receive, while maintaining the pool's mathematical balance and accounting for our 1% fee?

The Solution Logic

1. **Foundation: The Constant Product**

   The logic is built on the constant product formula `x * y = k`. We must solve for the output amount (`dy`) that keeps the product `k` constant after the swap.

2. **Fee Integration**

   We take a 1% fee on the *"input"* amount. This is elegantly implemented by multiplying the input by 99 and dividing by 100.

```
inputAmountWithFee = inputAmount * 99;
```

3. **The Implementation Formula**

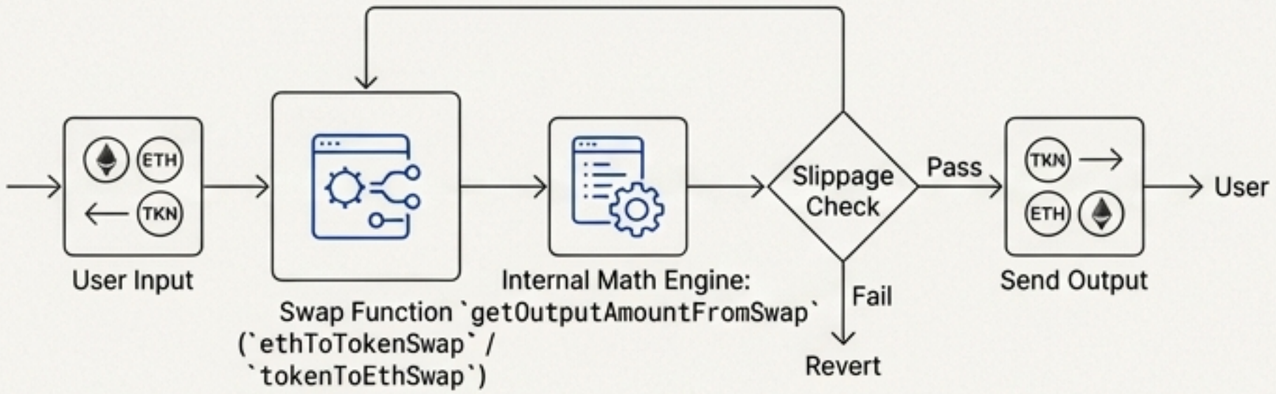
   The final calculation, derived from the constant product formula, ensures the invariant is preserved while accounting for the fee.

```
  numerator = inputAmountWithFee * outputReserve;
denominator = (inputReserve * 100) + inputAmountWithFee;
outputAmount = numerator / denominator;
```

Function Deep Dive: `ethToTokenSwap` & `tokenToEthSwap`

The Problem

How do we expose the core swap logic to users in a safe and practical way?



The Solution Logic

We architect two public-facing functions that serve as the user interface for swapping. They follow a clear, 4-step process:

1. Receive Input

Accept the input asset from the user (ETH via `msg.value` or TKN via `transferFrom`).

2. Calculate Output

Call our internal math engine, `getOutputAmountFromSwap`, to determine the output amount.

3. Check Slippage

Revert the transaction if the calculated output is less than a user-specified minimum (`min...ToReceive`). This protects users from unfavorable price changes during transaction execution.

Typically, a frontend application calculates the expected output and sets the minimum amount based on a user's chosen sllppage tolerance (e.g., 95% of expected).

4. Send Output

Transfer the calculated output asset to the user.

Code Implementation: Swap Functions

`ethToTokenSwap`

```
function ethToTokenSwap(uint256 minTokensToReceive) public
payable {
    uint256 tokenReserveBalance = getReserve();
    uint256 tokensToReceive = getOutputAmountFromSwap(
        msg.value,
        address(this).balance - msg.value,
        tokenReserveBalance
    );

    require(
        tokensToReceive >= minTokensToReceive,
        "Tokens received are less than minimum"
    );

    ERC20(tokenAddress).transfer(msg.sender, tokensToReceive);
}
```

`tokenToEthSwap`

```
function tokenToEthSwap(uint256 tokensToSwap, uint256
minEthToReceive) public {
    uint256 tokenReserveBalance = getReserve();
    uint256 ethToReceive = getOutputAmountFromSwap(
        tokensToSwap,
        tokenReserveBalance,
        address(this).balance
    );

    require(
        ethToReceive >= minEthToReceive,
        "ETH received is less than minimum"
    );

    ERC20(tokenAddress).transferFrom(
        msg.sender, address(this), tokensToSwap
    );

    payable(msg.sender).transfer(ethToReceive);
}
```

Step 4: Launching the Contracts

A clear, numbered sequence for deploying the contracts to a testnet like Sepolia.

1. Load Environment Variables

Ensure your terminal session has access to your secrets.

```
source .env
```

2. Deploy `Token.sol`

Launch the ERC-20 token contract first.

```
forge create --rpc-url $RPC_URL --private-key $PRIVATE_KEY \
    --etherscan-api-key $ETHERSCAN_API_KEY --verify \
    src/Token.sol:Token
```

Instruction: After running this, copy the deployed contract address. You will need it for the next step.

3. Deploy `Exchange.sol`

Launch the exchange, passing the token's address to its constructor.

```
forge create --rpc-url $RPC_URL --private-key $PRIVATE_KEY \
    --constructor-args <ADDRESS_OF_TOKEN_CONTRACT> \
    --etherscan-api-key $ETHERSCAN_API_KEY --verify \
    src/Exchange.sol:Exchange
```

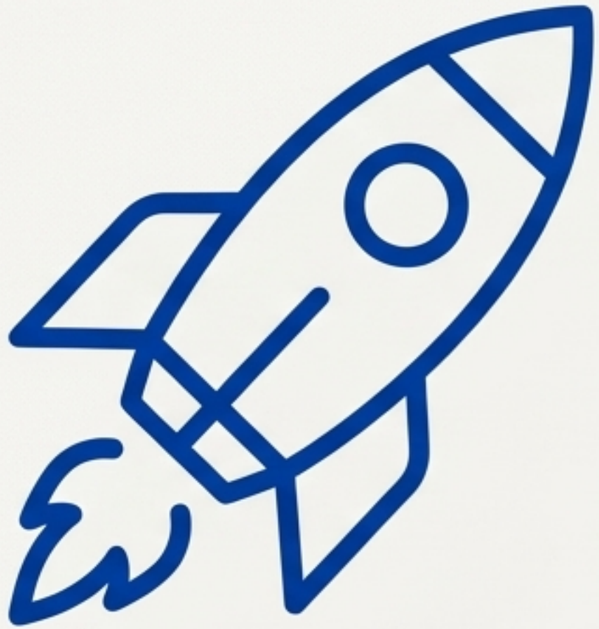
Paste the `Token.sol` contract address you just deployed here.

Post-Launch: Verification & Final Preparations

Pro Tip: Handling Verification Errors

You might see a message that your contract is "already verified," even if it isn't showing up on Etherscan. This happens when an identical contract has been verified before. To force verification for your specific deployment, use this command:

```
forge verify-contract <contract_address> <contract_name> --chain <chain_name>
```



Your DEX is Live!

Congratulations! Your `Token` and `Exchange` contracts are now live on the Sepolia testnet. Take note of both contract addresses, as we will need them soon.

What's Next?

In Part II, we will interact directly with our live contracts on Etherscan to add liquidity, perform swaps, and verify that our decentralized exchange works on liquidity, perform swaps, and verify that our decentralized exchange works perfectly.